# The RL Configuration

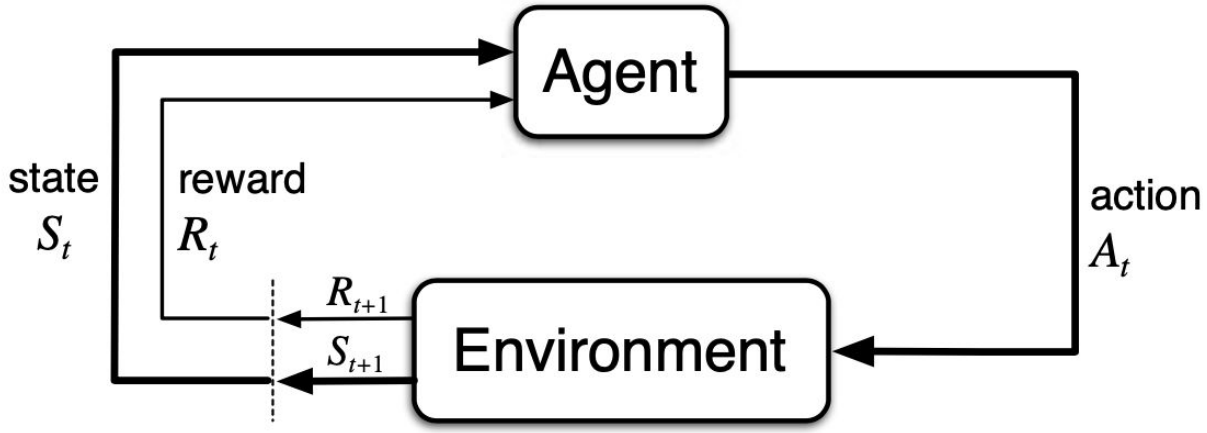



**Figure 3.1:** The agent–environment interaction in a Markov decision process.

Agent is the learner and the decision maker
Agent only performs actions
Environment rewards actions and indicates the new state

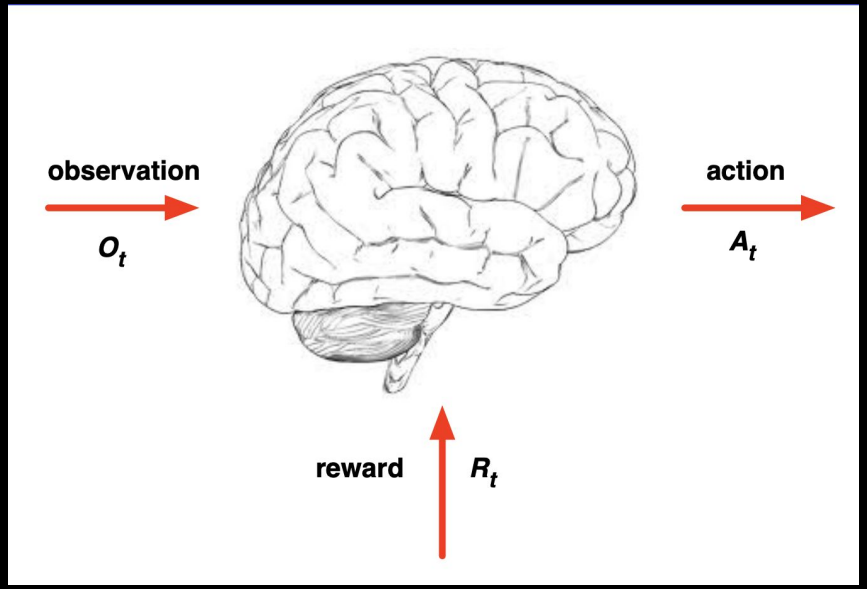
# Differentiator of RL

- There is no supervisor
- There is only a reward signal
- Feedback is delayed
- Agent’s actions affect the subsequent rewards, states, and actions

# Sequential Decision Making

- Actions have long term consequences
- Reward $R_t$ is a scalar *feedback signal* from the environment
- Reward indicates how well the agent is doing at step $t$
- Reward may be delayed
- Agent's job is to maximise the cumulative reward (in the future)
- Agent may be better to sacrifice immediate reward to gain more long-term reward

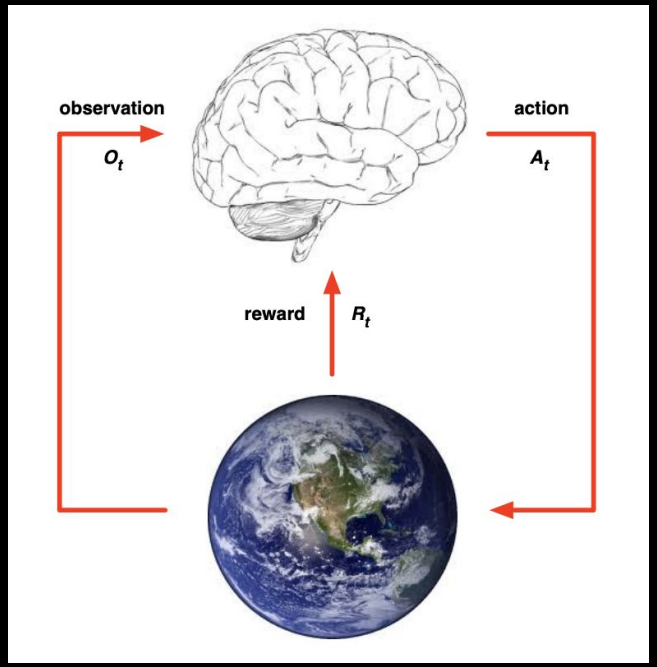
# Agent and Environment




- Agent
- Observation
- Action
- Reward

- Environment

# Agent and Environment




- At each step *t* the agent
  - Executes action $A_t$
  - Receives observation $O_t$
  - Receives scalar reward $R_t$

- The environment
  - Receives action $A_t$
  - Emits observation $O_{t+1}$
  - Emits scalar reward $R_{t+1}$
  - $t$ increments at environment step

# History and State

The history is the sequence of observations, actions, rewards

$$H_t = O_1, R_1, A_1, \ldots, A_{t-1}, O_t, R_t$$

- all observable variables up to time *t*
- the sensorimotor stream of a robot or embodied agent

What happens next depends on the history

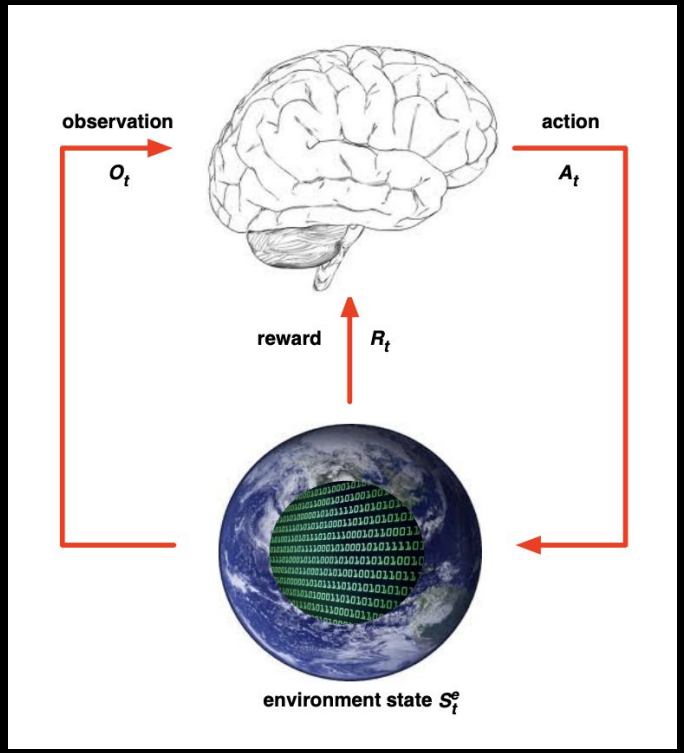
- The agent selects actions
- The environment selects observations/rewards

State is the information used to determine what happens next

Formally, state is a function of the history

$$S_t = f(H_t)$$

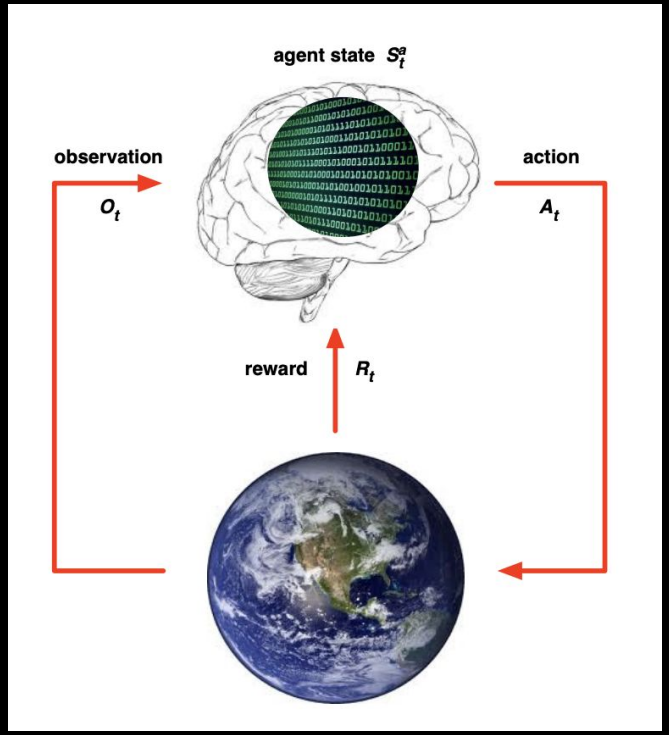
# Environment State




The *environment state* $S^e_t$ is the environment's private representation

- whatever data the environment uses to pick the next observation/reward
- The environment state is not usually visible to the agent
- Even if $S^e_t$ is visible, it may contain irrelevant information

# Agent State




The *agent state* $S^a_t$ is the agent's internal representation

- whatever information the agent uses to pick the next action.
- it is the information used by reinforcement learning algorithms.
- It can be any function of history

$$S^a_t = f(H_t)$$

# Information State

- An *information state*, *aka Markov state*, contains all useful information from the history.

- A state $S_t$ is Markov if and only if

$$\mathbb{P}[S_{t+1} | S_t] = \mathbb{P}[S_{t+1} | S_1, \ldots, S_t]$$

- The future is independent of the past given the present

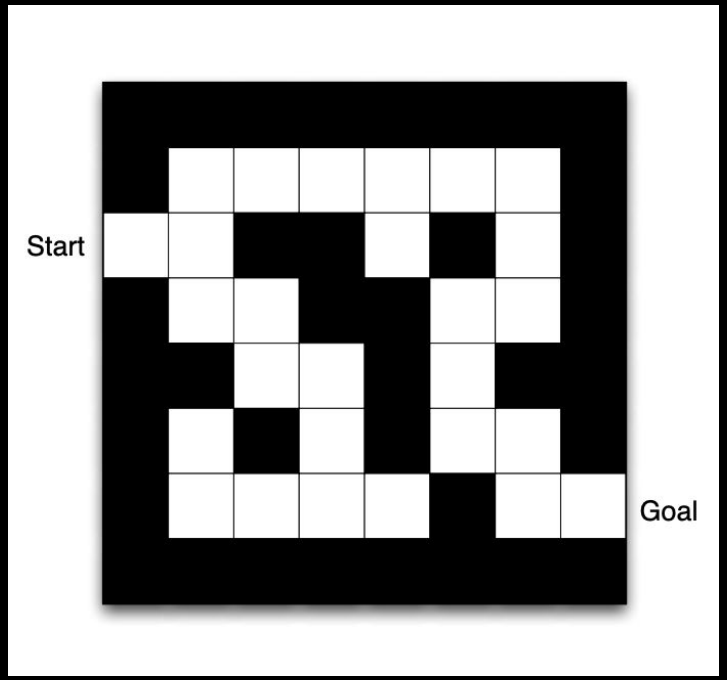
Once the state is known, the history may be thrown away

- The state is a sufficient *statistic* of the future
- The environment state $S_t^e$ is *Markov*
- The history $H_t$ is *Markov*

*Statistic*

- a fact or data or evidence.

# Markov States in Maze Example



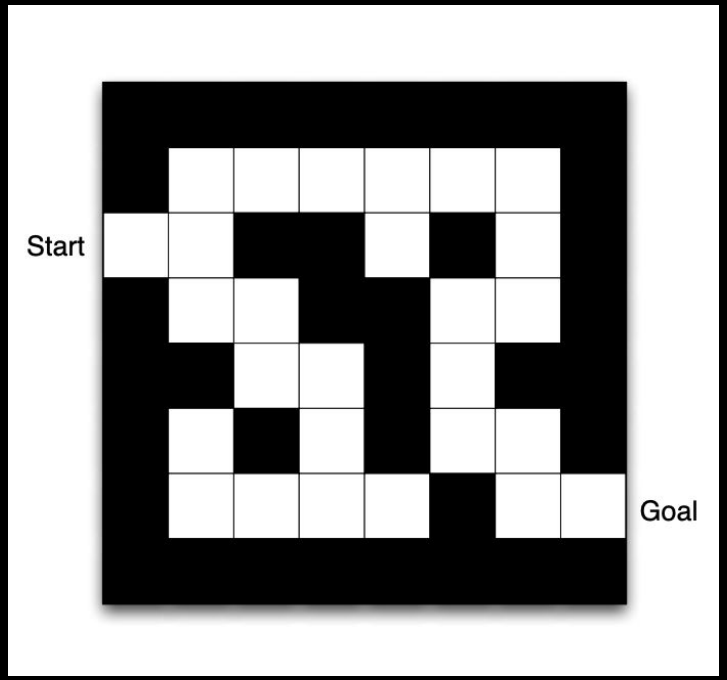

What is the environment?

What is the state space?

What is the action space?

What may be the rewards?

# Markov States in Maze Example

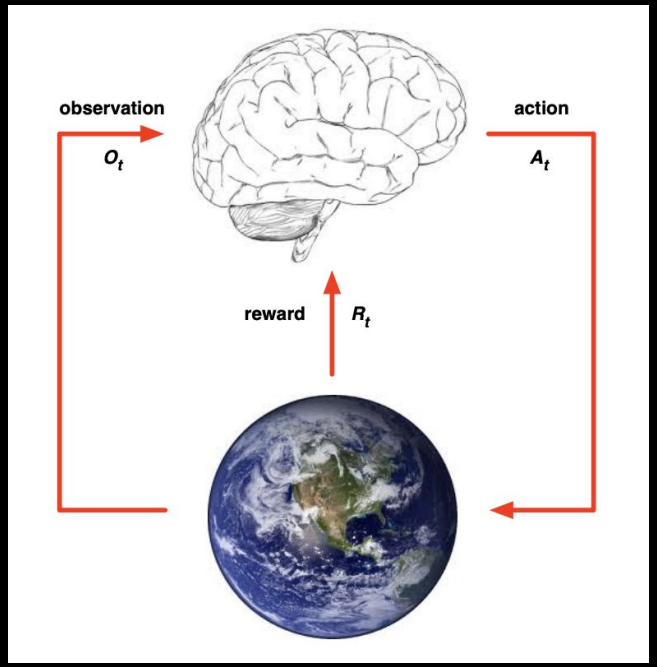



Rewards: -1 per time-step

Actions: *N, E, S, W*

States: Agent's location

# Fully Observable Environments




Full observability

- agent directly observes environment state
- $O_t = S^a_t = S^e_t$

Agent state = environment state = information state

Formally, this is a *Markov decision process* (MDP)

# Partially Observable Environments

Agent indirectly observes environment

- A trading agent only observes current prices
- A robot with camera vision isn't told its absolute location
- Agent state ≠ Environment state
- Formally this is a partially observable Markov decision process (POMDP) Agent must construct its own state representation $S^a_t$
  - Complete history: $S^a_t = H_t$
  - ***Beliefs*** of environment state: $S^a_t = (\mathbb{P}[S^e_t = s_1], \ldots, \mathbb{P}[S^e_t = s_n])$
  - Recurrent neural network: $S^a_t = \sigma(S^a_{t-1} W_s + O_t W_o)$

# Major Components of an RL Agent

An RL agent may include one or more of these components

- Policy
  - agent’s behaviour function
- Reward Signal
  - A scalar value
- Value function
  - how good is each state and/or action
- Model
  - agent’s representation of the environment

# Policy

A policy is the agent's behaviour in a state

- May involve computationally extensive search or a cheap table lookup
- May be stochastic, specifying probabilities for each action
- Choosing from a set of actions in a state.

It is a map from state to action

- Deterministic policy: $a = \pi(s)$
- Stochastic policy: $\pi(a|s) = \mathbb{P}[A_t = a|S_t = s]$

# Reward

- Reward Signal
  - A single number supplied by the environment
  - immediate  intrinsic desirability of environment states
  - Agent tries to maximize the total reward it receives over a long run
  - Reward signal is the primary basis for altering the policy
  - Reward signals may be stochastic functions of the state of the environment and the actions taken

# Value Function

- Value Function
  - Specifies what is good in the long run
  - Value of a state is the total reward the agent can expect to accumulate in the future starting from that state
  - Long-term desirability of states
  - Takes into account states that are likely to follow and rewards available in those states
- A state may *always* yield low reward and yet may have high value
  - It is regularly followed by other states that yield high rewards
- The opposite may also be true!

# Value Function

Value function is a prediction of future reward

Used to evaluate the goodness/badness of states

Therefore used to select among actions

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \mid St = s]$$

*where* $\gamma$ is the *discount factor*.

# Model

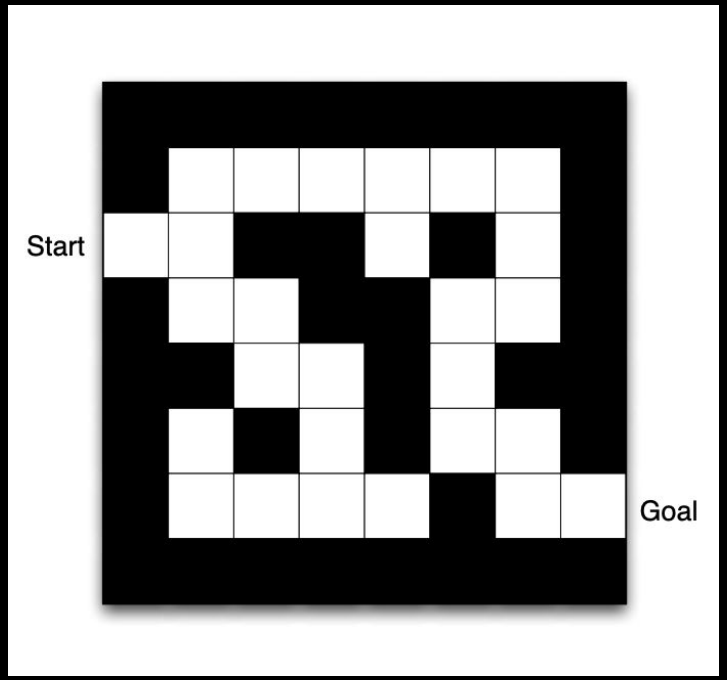
A *model* predicts what the environment will do next

- P predicts the next state
- R predicts the next (immediate) reward, e.g

$$P^a_{ss'} = \mathbb{P}[S_{t+1} = s' | S_t = s, A_t = a]$$

$$R^a_s = \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$$

# Markov States in Maze Example

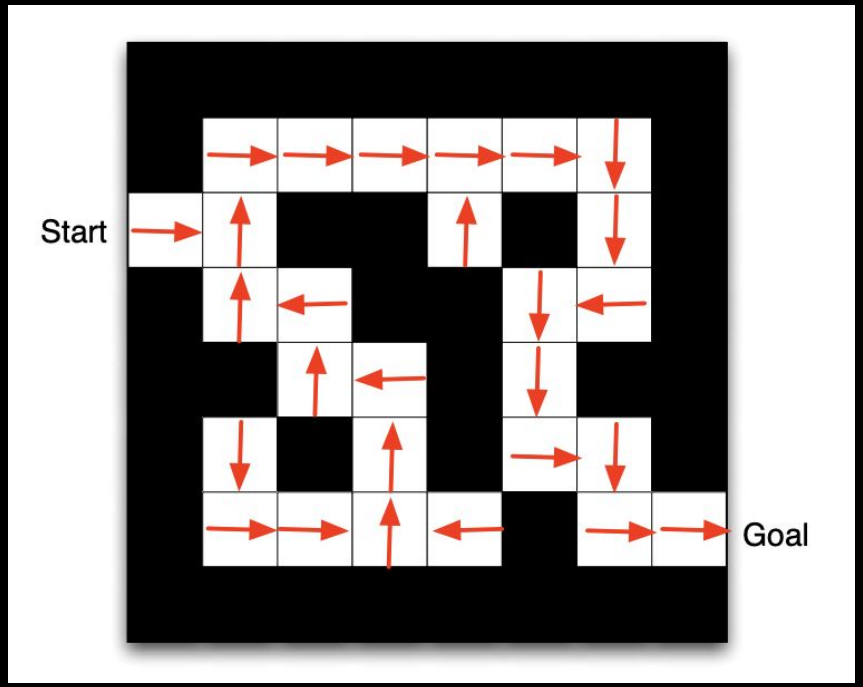



Rewards: -1 per time-step

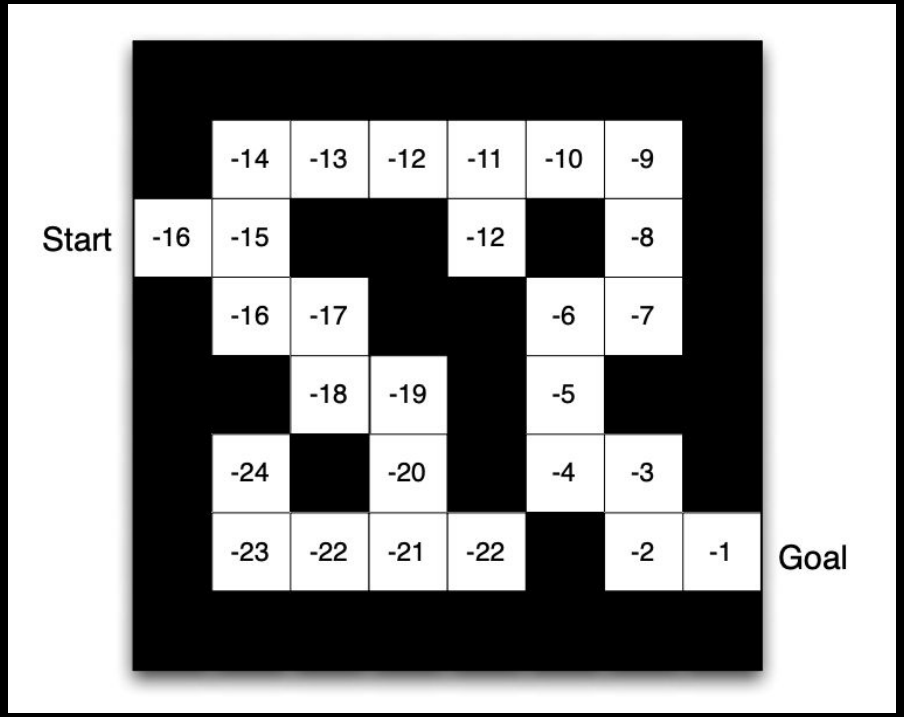
Actions: *N, E, S, W*

States: Agent's location

# Maze Policy

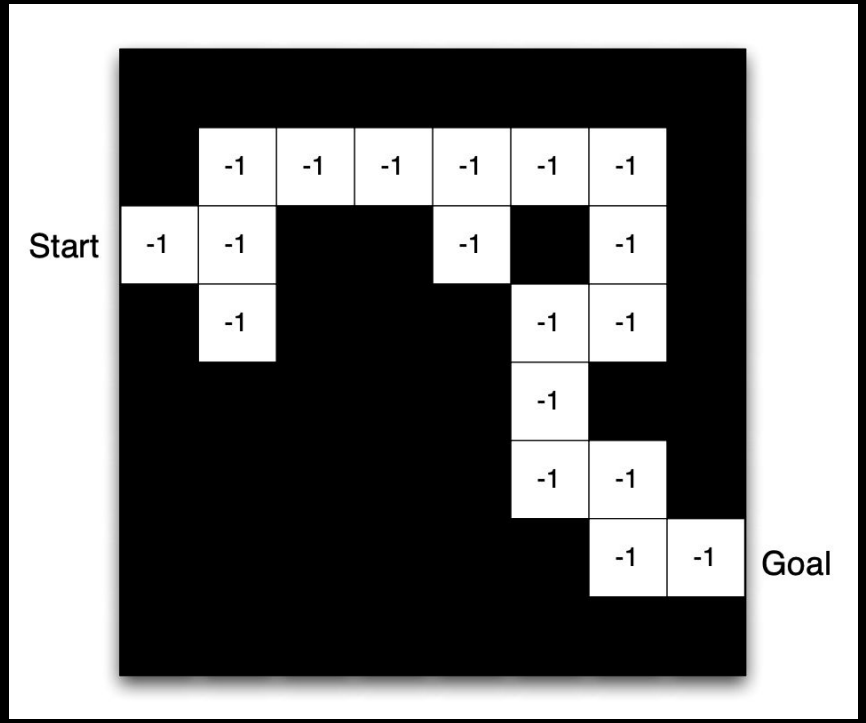



Arrows represent policy $\pi(s)$ for each state $s$




Numbers represent value $v_\pi(s)$ of each state $s$

# Maze Model



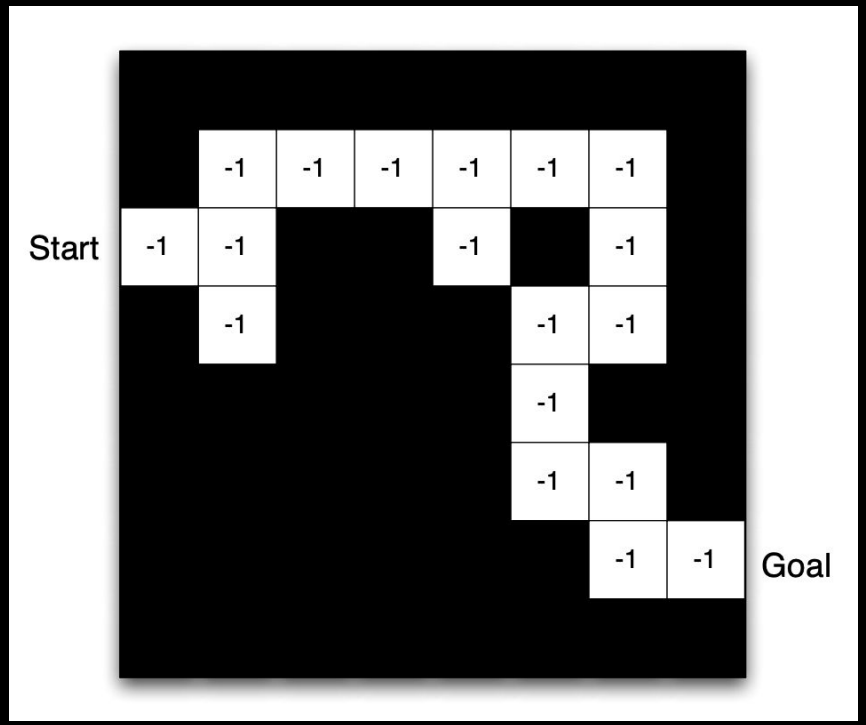

Agent may have an internal model of the environment

Dynamics
- how actions change the state

Rewards
- how much reward from each state

The model may be imperfect

# Maze Model




Grid layout represents transition model $P^a_{ss'}$

Numbers represent immediate reward $R^a_s$ from each state $s$ (same for all a)

# Categorizing Agents

Model Free

- Policy and/or Value Function
- No Model

Model Based

- Policy and/or Value Function
- Model

# Categorizing Agents

Value Based

- No Policy (Implicit)
- Value Function

Policy Based

- Policy
- No Value Function

Actor Critic

- Policy
- Value Function

# Exploration and Exploitation - 1

Reinforcement learning is like trial-and-error learning

The agent should discover a good policy

From its experiences of the environment

Without losing too much reward along the way

# Exploration and Exploitation - 2

*Exploration* finds more information about the environment

*Exploitation* exploits known information to maximise reward

It is usually important to *explore* as well as *exploit*